TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 5
Persistencia y estado

Clave-valor · SQLite · Estado global · Conectividad

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 5

Qué vamos a ver

Objetivos

- Elegir dónde guardar cada dato.
- Usar SQLite con la API moderna y migraciones.
- Compartir estado entre pantallas sin sufrir.
- Que la app siga siendo útil sin conexión.

Partes

- Qué guardar y dónde
- Clave-valor
- SQLite
- Estado global
- Conectividad

UNIDAD 5 · PARTE 1

Qué guardar y dónde

Cuatro lugares, cuatro problemas distintos.

Los cuatro lugares

Clave-valor

`expo-sqlite/kv-store`

Preferencias, el último filtro elegido, si ya vio el tutorial. Poco dato, sin consultas.

Base relacional

`expo-sqlite`

Muchos registros con relaciones y consultas. Trabajo sin conexión de verdad.

Secretos

`expo-secure-store`

Tokens, claves. Cifrado por el sistema. Va en la Unidad 6.

Archivos

`expo-file-system`

Fotos, PDF, adjuntos. Lo vimos en la Unidad 4.

> Nada de esto es la respuesta a «¿dónde guardo la lista de productos que trae la API?». Eso es **caché de estado del servidor** y lo resolvió el deck anterior. Persistir a mano lo que TanStack Query ya cachea es trabajo doble.

UNIDAD 5 · PARTE 2

Clave-valor

Lo más simple que hay, y lo que cambió.

UNIDAD 5 · PARTE 2 · CLAVE-VALOR

De AsyncStorage a `kv-store`

ANTES — LO QUE DICE EL APUNTE VIEJO

```
import AsyncStorage
  from '@react-native-async-storage/async-storage';

await AsyncStorage.setItem('tema', 'oscuro');
const tema = await AsyncStorage.getItem('tema');
```

AHORA — LO QUE USAMOS

```
import Storage from 'expo-sqlite/kv-store';

await Storage.setItem('tema', 'oscuro');
const tema = await Storage.getItem('tema');
```

Misma API

Migrar es cambiar la línea del `import`. Nada más.

Una dependencia menos

Si ya usás `expo-sqlite`, esto viene incluido.

Y encima sincrónico

`getItemSync()` y `setItemSync()`, que AsyncStorage no tiene.

Por qué importa lo sincrónico

ASINCRÓNICO

```
const [tema, setTema] = useState(null);

useEffect(() => {
  Storage.getItem('tema').then(setTema);
}, []);

if (tema === null) return <Cargando />;
```

Un parpadeo con el tema equivocado antes de que resuelva la promesa.

SINCRÓNICO

```
const [tema, setTema] = useState(
  () => Storage.getItemSync('tema') ?? 'claro'
);
```

El valor correcto ya en el primer render. Sin estado de carga.

Para cosas chicas que se leen al arrancar —tema, idioma, si ya inició sesión— la versión sincrónica elimina toda una clase de parpadeos.

Solo guarda texto

```
import Storage from 'expo-sqlite/kv-store';

// Guardar un objeto: hay que serializarlo.
await Storage.setItem('usuario', JSON.stringify({ id: 7, nombre: 'Ana' }));

// Leerlo: puede no estar. getItem devuelve string | null.
const crudo = await Storage.getItem('usuario');
const usuario = crudo ? JSON.parse(crudo) : null;

await Storage.removeItem('usuario');
```

El error clásico

`JSON.parse(null)` devuelve `null`, pero `JSON.parse('')` revienta. Y un número guardado vuelve como string.

La costumbre sana

Envolvelo en dos funciones, `guardar()` y `leer()`, con el `JSON` y el `try/catch` adentro. Una vez.

Lo que NO va acá

Tokens y contraseñas

No está cifrado. En un dispositivo con root se lee entero.

→ `expo-secure-store`

Listas largas

No hay consultas ni índices: para filtrar hay que leer todo y recorrerlo en JavaScript.

→ `expo-sqlite`

Imágenes en base64

Infla la base y la vuelve lenta. Se guarda el archivo y acá, a lo sumo, la ruta.

→ `expo-file-system`

> Guardar el token de sesión en almacenamiento común es el hallazgo más frecuente en los TP. Funciona, y es exactamente el punto que la Unidad 6 va a marcar como vulnerabilidad.

UNIDAD 5 · PARTE 3

SQLite

Una base relacional entera adentro del teléfono.

Cuándo hace falta

Sí

- La app tiene que funcionar sin conexión y no solo mostrar una caché.
- Hay cientos o miles de registros.
- Hace falta filtrar, ordenar, agrupar o cruzar tablas.
- Los datos tienen relaciones: pedidos con ítems, cursos con alumnos.

No

- Son cuatro preferencias.
- Es una copia de lo que trae la API y siempre hay internet.
- Es un solo objeto que se lee entero cada vez.

Una base sin consultas es un archivo de texto caro.

Ventaja para el TP: es SQL. Lo que ya saben de bases de datos se aplica tal cual — `CREATE TABLE`, `JOIN`, `WHERE`, índices.

La API moderna: proveedor y hook

```tsx
// app/_layout.tsx  ·  import { SQLiteProvider } from 'expo-sqlite'
export default function Layout() {
  return (
    <SQLiteProvider databaseName="app.db" onInit={migrar}>
      <Slot />
    </SQLiteProvider>
  );
}
```

```tsx
// en cualquier pantalla  ·  import { useSQLiteContext } from 'expo-sqlite'
const db = useSQLiteContext();
const tareas = await db.getAllAsync('SELECT * FROM tareas');
```

Una sola conexión para toda la app, abierta una vez y compartida por contexto. `onInit` corre **antes** de que se renderice nada adentro: cuando tu pantalla consulta, las tablas ya existen.

Cuatro métodos, cuatro usos

`execAsync(sql)`

Varias sentencias de una. Para crear el esquema.

No escapa parámetros: nunca con datos del usuario.

`runAsync(sql, params)`

`INSERT`, `UPDATE`, `DELETE`. Devuelve `lastInsertRowId` y `changes`.

`getFirstAsync(sql, params)`

Una fila o `null`. Buscar por id, contar.

`getAllAsync(sql, params)`

Un array de objetos. La lista que va a la pantalla.

> Todos tienen gemelo sincrónico — `runSync`, `getAllSync` —. Andan, pero **bloquean el hilo de JS**: sobre una tabla grande se nota como tirones al hacer scroll. Usá los asincrónicos salvo que midas otra cosa.

Parámetros, siempre

NUNCA

```
await db.execAsync(
  `SELECT * FROM usuarios
   WHERE nombre = '${entrada}'`
);

// entrada = "'; DROP TABLE usuarios; --"
```

SIEMPRE

```
await db.getAllAsync(
  'SELECT * FROM usuarios WHERE nombre = ?',
  entrada
);

await db.runAsync(
  'UPDATE tareas SET hecha = ? WHERE id = $id',
  [1], { $id: 7 }
);
```

El `?` no es una comodidad de sintaxis: SQLite recibe la consulta y los valores **por separado**, así que el texto del usuario no puede convertirse en instrucción. Es la defensa, no una ayuda.

Migraciones

```
async function migrar(db: SQLiteDatabase) {
  const VERSION = 1;
  let { user_version: actual } = await db.getFirstAsync('PRAGMA user_version');
  if (actual >= VERSION) return;

  if (actual === 0) {
    await db.execAsync(`PRAGMA journal_mode = 'wal';
      CREATE TABLE tareas (id INTEGER PRIMARY KEY NOT NULL, titulo TEXT NOT NULL,
                           hecha INTEGER NOT NULL DEFAULT 0);`);
    actual = 1;
  }
  // if (actual === 1) { …la próxima migración va acá… }

  await db.execAsync(`PRAGMA user_version = ${VERSION}`);
}
```

La base **sobrevive a la actualización de la app**. Si agregás una columna sin escribir la migración, a quien ya la tenía instalada le explota — y en tu teléfono, que lo reinstalaste, anda perfecto.

Transacciones

```
await db.withTransactionAsync(async () => {
  const { lastInsertRowId } = await db.runAsync(
    'INSERT INTO pedidos (cliente, fecha) VALUES (?, ?)', cliente, fecha,
  );

  for (const item of items) {
    await db.runAsync(
      'INSERT INTO items (pedido_id, producto, cantidad) VALUES (?, ?, ?)',
      lastInsertRowId, item.producto, item.cantidad,
    );
  }
});
```

O entra todo o no entra nada. Sin transacción, una app que se cierra a mitad del bucle deja un pedido sin ítems — y esa fila rota queda para siempre.

Además es mucho más rápido: mil `INSERT` sueltos hacen mil escrituras a disco; dentro de una transacción, una sola.

UNIDAD 5 · PARTE 4

Estado global

Lo que varias pantallas necesitan al mismo tiempo.

El problema

Pasar props hacia abajo

El carrito vive en la pantalla de inicio y lo necesita un botón cuatro niveles más abajo. Cada componente del medio recibe y reenvía algo que no usa.

Context como estado

Resuelve el paso de props, pero **cuando cambia el valor se vuelve a renderizar todo el árbol**, incluso lo que solo lee una parte.

> Context sigue siendo lo correcto para cosas que **casi no cambian** —el tema, el idioma, la sesión—. Para lo que cambia seguido conviene algo que permita suscribirse a un pedacito.

Antes de agregar una librería

`useState`
Si el dato lo usa una pantalla y sus hijos directos. La mayoría de los casos.

Context
Si lo leen muchas pantallas pero **cambia poco**: tema, idioma, sesión. Viene con React.

TanStack Query
Si el dato viene del servidor. **Ya es global**: la caché se comparte entre pantallas sin que hagas nada.

Zustand
Si es estado del cliente, cambia seguido y lo tocan pantallas que no son parientes.

> El error de diseño más común es meter en el estado global la lista que trae la API. Eso ya lo resolvió TanStack Query; duplicarlo es garantizar que las dos copias se desincronicen.

Context + Provider: sin librerías

```tsx
// contexto/carrito.tsx
import { createContext, use, useState, type ReactNode } from 'react';

type Carrito = { items: Item[]; agregar: (i: Item) => void; quitar: (id: string) => void };
const CarritoContext = createContext<Carrito | null>(null);

export function CarritoProvider({ children }: { children: ReactNode }) {
  const [items, setItems] = useState<Item[]>([]);
  const agregar = (item: Item) => setItems((s) => [...s, item]);
  const quitar = (id: string) => setItems((s) => s.filter((i) => i.id !== id));

  return (
    <CarritoContext value={{ items, agregar, quitar }}>
      {children}
    </CarritoContext>
  );
}
```

> El mismo patrón que `SQLiteProvider + useSQLiteContext` de la Parte 3: **un componente arriba guarda el valor** y un hook abajo lo lee.

Envolver la app y leerlo

EL HOOK — MISMO ARCHIVO

```
export function useCarrito() {
  const carrito = use(CarritoContext);
  if (!carrito) {
    throw new Error('Falta <CarritoProvider>');
  }
  return carrito;
}
```

EL PROVEEDOR — APP/_LAYOUT.TSX

```
export default function Layout() {
  return (
    <CarritoProvider>
      <Slot />
    </CarritoProvider>
  );
}
```

Desde cualquier pantalla: `const { items, agregar } = useCarrito();` Sin pasar props y sin instalar nada.

El costo: cuando cambia `items` se vuelven a renderizar **todos los componentes que llaman a `useCarrito()`**, aunque solo usen `agregar`. Context entrega el valor entero; no hay forma de suscribirse a un pedazo.

Zustand: el store

```ts
// store/carrito.ts
import { create } from 'zustand';

type Carrito = {
  items: Item[];
  agregar: (item: Item) => void;
  quitar: (id: string) => void;
  total: () => number;
};

export const useCarrito = create<Carrito>((set, get) => ({
  items: [],
  agregar: (item) => set((s) => ({ items: [...s.items, item] })),
  quitar: (id) => set((s) => ({ items: s.items.filter(i => i.id !== id) })),
  total: () => get().items.reduce((a, i) => a + i.precio * i.cantidad, 0),
}));
```

> El store **es un hook**. No hay proveedor que envolver, no hay reducer, no hay acciones ni tipos de acción: estado y funciones que lo cambian, en el mismo objeto.

Usarlo, y el detalle del selector

MAL — SE LLEVA TODO

```
const { items, agregar } = useCarrito();

// Este componente se vuelve a renderizar
// ante CUALQUIER cambio del store.
```

BIEN — UN SELECTOR POR DATO

```
const cantidad = useCarrito(s => s.items.length);
const agregar  = useCarrito(s => s.agregar);

// Solo se vuelve a renderizar cuando
// cambia la cantidad. La función es estable.
```

Esa es la ventaja concreta sobre Context: **te suscribís a un pedazo del estado, no al estado entero**. El ícono del carrito en el encabezado se actualiza sin tocar el resto de la pantalla.

Que sobreviva al cierre de la app

```
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import Storage from 'expo-sqlite/kv-store';

export const useCarrito = create<Carrito>()(
  persist(
    (set, get) => ({ /* …lo mismo de antes… */ }),
    {
      name: 'carrito',
      storage: createJSONStorage(() => Storage),
      partialize: (s) => ({ items: s.items }),   // guardar solo los datos
    },
  ),
);
```

Acá se cierra el círculo de las dos primeras partes: el estado global se guarda solo, en el mismo `kv-store`. `partialize` evita persistir las funciones, que no tiene sentido serializar.

UNIDAD 5 · PARTE 5

Conectividad

Saber que no hay red, y hacer algo al respecto.

expo-network

```
import { useNetworkState, NetworkStateType } from 'expo-network';

export function AvisoDeRed() {
  const red = useNetworkState();

  if (red.isInternetReachable === false) {
    return <Barra texto="Sin conexión. Vas a ver datos guardados." />;
  }

  if (red.type === NetworkStateType.CELLULAR) {
    return <Barra texto="Estás usando datos móviles." />;
  }

  return null;
}
```

`isConnected` y `isInternetReachable` no son lo mismo. El Wi-Fi del bar que te pide iniciar sesión da `isConnected: true` y ningún dato pasa. **El que importa es el segundo.**

Diseñar para que se corte

Al leer

- Mostrar lo último que tenés, avisando que es viejo.
- Nunca una pantalla en blanco.
- Que el reintento sea un botón, no un misterio.

Al escribir

- Guardar local y marcar como pendiente.
- Sincronizar cuando vuelva la red.
- Decir con claridad qué se envió y qué no.

Ese «guardar y sincronizar después» es exactamente para lo que sirve SQLite: una tabla de pendientes con su estado, y un proceso que la vacía cuando `isInternetReachable` vuelve a ser `true` .

Se evalúa en el TP integrador: **poné el teléfono en modo avión y usá tu app**. Si queda colgada en un spinner, falta trabajo.

UNIDAD 5

Documentación

- `docs.expo.dev/versions/latest/sdk/sqlite`
- `docs.expo.dev/versions/latest/sdk/sqlite/#key-value-storage`
- `docs.expo.dev/versions/latest/sdk/network`
- `zustand.docs.pmnd.rs`